
Data Structures

(Linked List and Stack)

Introduction

Faculty Name: Naeem Ahmed (NMA)

Room: UB80601

Consultation Hours:

Sunday: 9:30 AM - 11:00 AM

Monday: 11:00 AM-12:30 PM

Wednesday: 11:00 AM-2:00 PM

Linked List

Storage system of computer

Speed:

CPU > RAM > DISK

Memory Management Technique

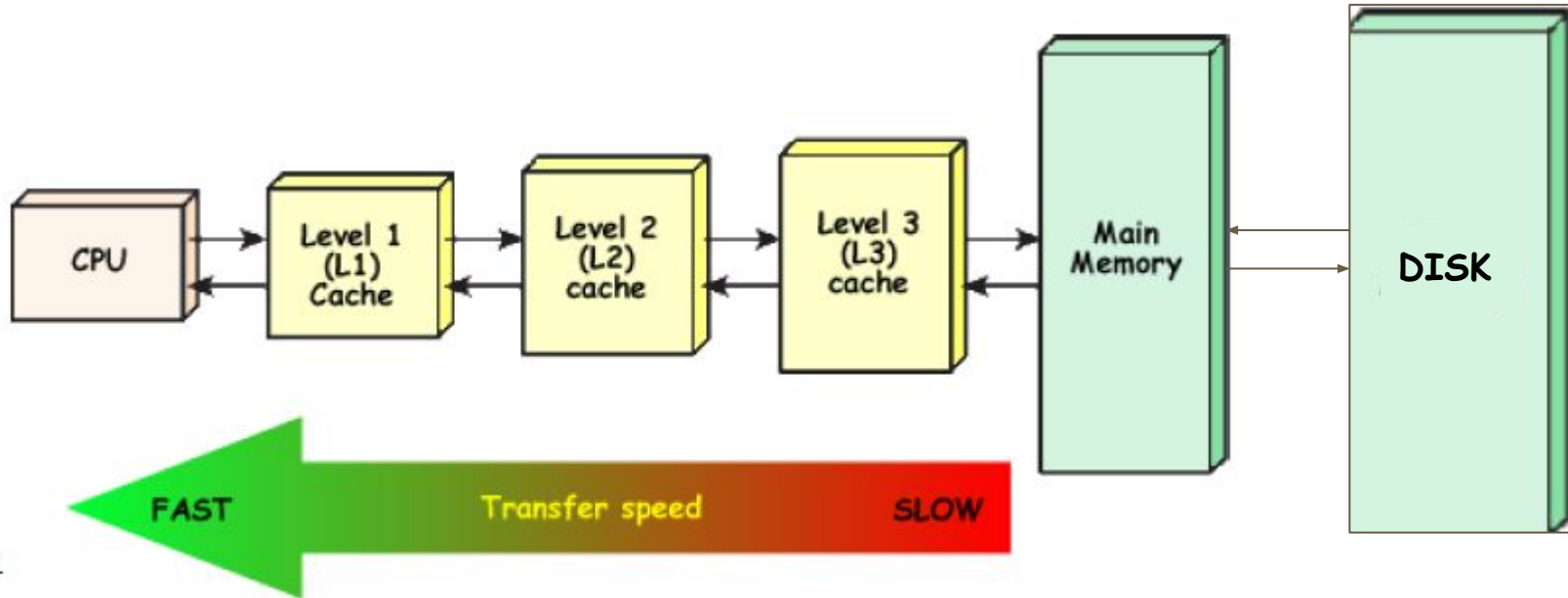
- Computations are held at CPU
- CPU needs data
- RAM loads data from DISK for CPU
- RAM loads a large block of data at once, which is several KB or MB, from the storage even if the CPU asks for a single byte of data.
- Assumption: CPU will ask for more data from locations nearby to its first request

Memory Management Technique

- Smaller memory unit tends to be faster
- Slowness of the RAM (no longer the slowness of the disk, which has been taken care of) hurts overall program performance greatly.
- So they just reused the same idea by introducing something called caches.
- Caches are like RAM but much smaller and faster.
- So when a program runs, if the CPU ask for a single byte of data that is available in the RAM, the hardware actually load multiple consecutive bytes, at the range of 256 bytes or more from the RAM to the cache.

Memory Management Technique

This idea of loading more data in closer memory works so good that now a days, all computers has multiple levels of caches between the CPU and the RAM.



Effect on Performance

- **Array:** Elements are allocated in consecutive memory cells . So all are in the RAM or Cache.
Traversal is extremely fast.
Just occasional data transfers between the cache and the RAM.
- **Linked List:** Space for a list element is allocated when you add the element to the list.
So, the elements are non-consecutive.
The hardware has to fetch data from the RAM to caches many times more.
Traversal is very slow.

Linked List Issue

- We can only move forward.
- Can't traverse backward.
- This type of Linked list only holds one link per node. From one node to the next.
- Called Singly Linear linked list.

Linked List Types

Category 1	Category 2	Category 3
Non-Dummy Headed	Singly	Linear
Dummy Headed	Doubly	Circular

Linked List Types

Non-Dummy Headed: It means that the linked list's first node (the head) contains an item along with the information of the next node. For example, the lists we have used till now are Non-Dummy headed.

Example: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$. In this Linked list, the first node stores the value head.

Linked List Types

Dummy-Headed: This refers to the linked list where a head node is a node-type object but it does not store any element on its own. Rather it stores the information of the next node where the starting value is stored. The benefit of this is that we do not need to handle the first item of the data carefully now. To illustrate, we can remove or add items at beginning of the list without handling the head delicately. The reason is the first item is stored after the dummy head.

Example: $DH \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 40$. In this linked list, DH refers dummy head which is a node without any element. The first item is stored after the dummy head.

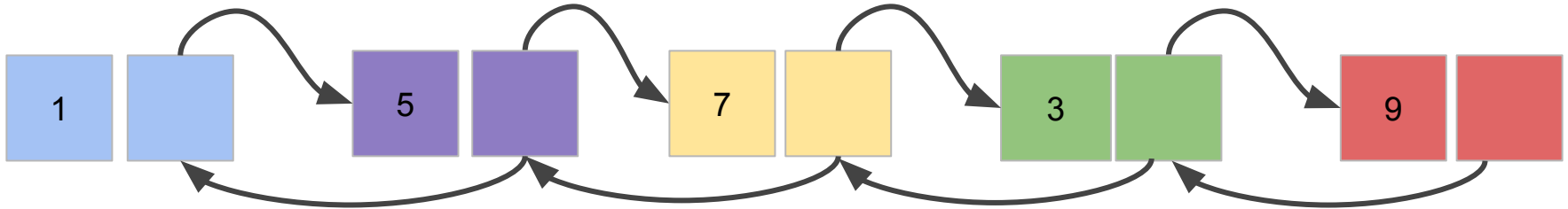
Linked List Types

Singly: It means every node has only the information of its next node. The reason is the design of the Node class has only one variable (next variable). Up until now all the list we worked on is Singly Linked List.



Linked List Types

Doubly: It means a node knows its next node and its previous node's information. To achieve this we need to modify the Node class design where we store the previous node's information. Example of Doubly Node class in python



Linked List Types

Doubly Linked List Class in Python:

```
class DoublyNode:
    def __init__(self, elem, next, prev):
        self.elem = elem
        self.next = next # To store the next node's reference.
        self.prev = prev # To store the previous node's reference.
```

DoublyNode

elem

prev

next

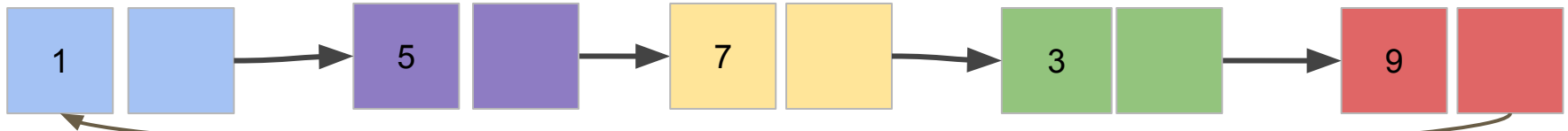
Linked List Types

Linear: Linear linked list refers to the linked list where the linked list is linear in structure. To illustrate, the last node of the linked list will refer to None.



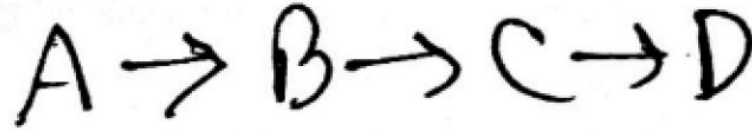
Linked List Types

Circular: Circular linked list refers to a linked list where the linked list is circular in structure. To illustrate, the last node of the linked list will refer to the starting node.

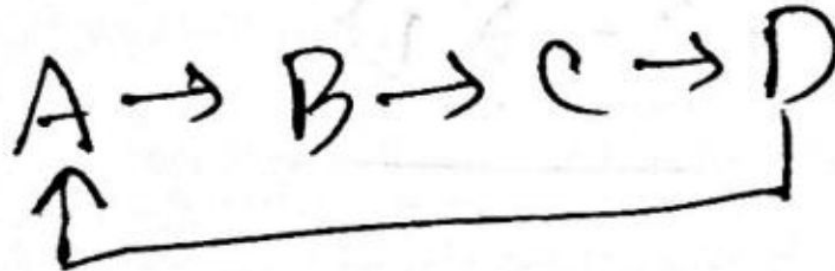


Linked List Types

1. Non-Dummy Headed Singly Linear Linked List:

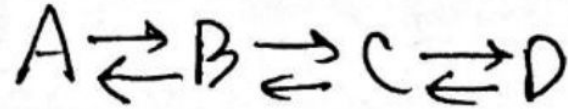


2. Non-Dummy Headed Singly Circular Linked List:

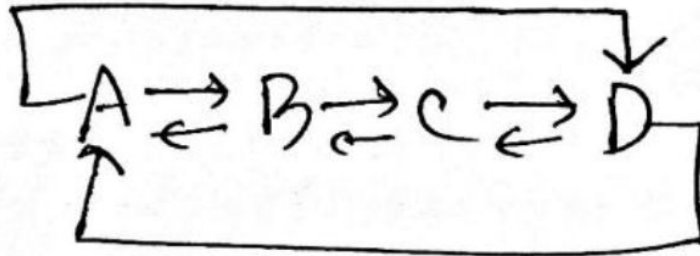


Linked List Types

3. Non-Dummy Headed Doubly Linear Linked List:

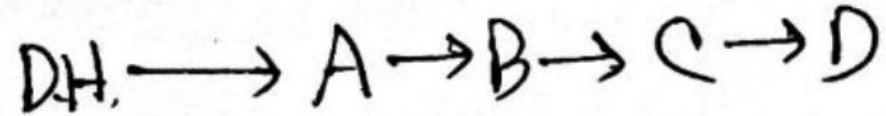


4. Non-Dummy Headed Doubly Circular Linked List:

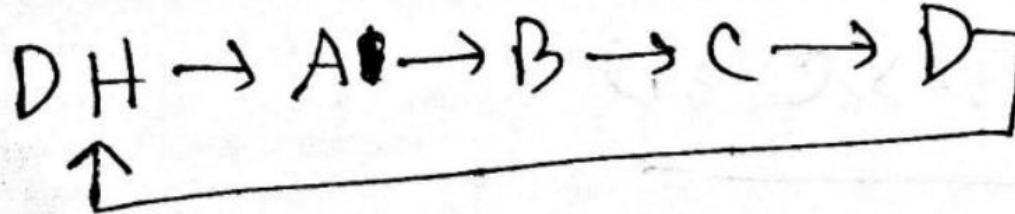


Linked List Types

5. Dummy Headed Singly Linear Linked List:



6. Dummy Headed Singly Circular Linked List:

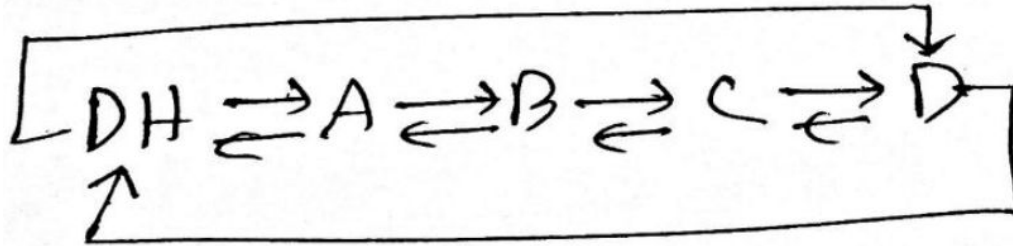


Linked List Types

7. Dummy Headed Doubly Linear Linked List:

$DH \rightleftarrows A \rightleftarrows B \rightleftarrows C \rightleftarrows D$

8. Dummy Headed Doubly Circular Linked List:



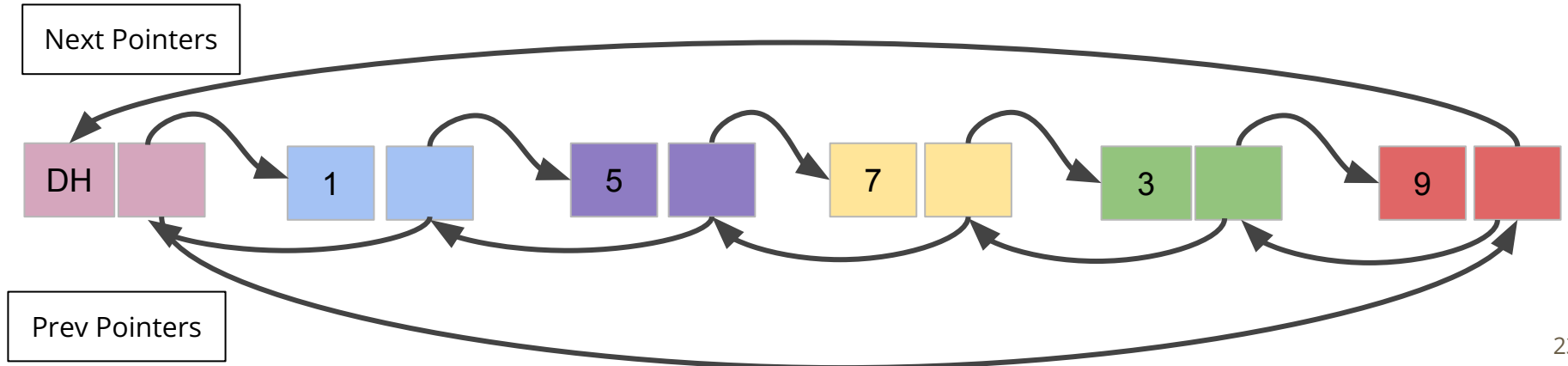
Dummy Headed Doubly Circular Linked List



Dummy Headed Doubly Circular Linked List

The list will have

- a dummy head
- doubly connection
- circular structure



The Node Class

Will have doubly connection, ie two way to go from current node.

```
class DoublyNode:
    def __init__(self, elem, next, prev):
        self.elem = elem
        self.next = next # To store the next node's reference.
        self.prev = prev # To store the previous node's reference.
```


Creation of the List

It is similar to creating a linked list. The difference is that we need to ensure the connections in a proper way.

The below function takes an array and creates a dummy-headed doubly circular linked list.

```
1 def createList(a):
2     dh = DoublyNode(None, None, None)
3     dh.next = dh
4     dh.prev = dh
5     tail = dh
6
7     for i in range(len(a)):
8         n = DoublyNode(a[i], dh, tail)
9         tail.next = n
10        tail = tail.next
11        dh.prev = tail
12
13    return dh
```

Iteration

It is similar to the previous linked list. The difference is that it will start from the next item of the dummy head and run till the pointer comes back to the dummy head.

```
1 def iteration(dh):  
2     temp = dh.next  
3     while temp != dh:  
4         print(temp.elem)  
5         temp = temp.next
```

NodeAt

The nodeAt method is similar to the previous one. The difference is that the dummy head does not represent any index as it does not have any value.

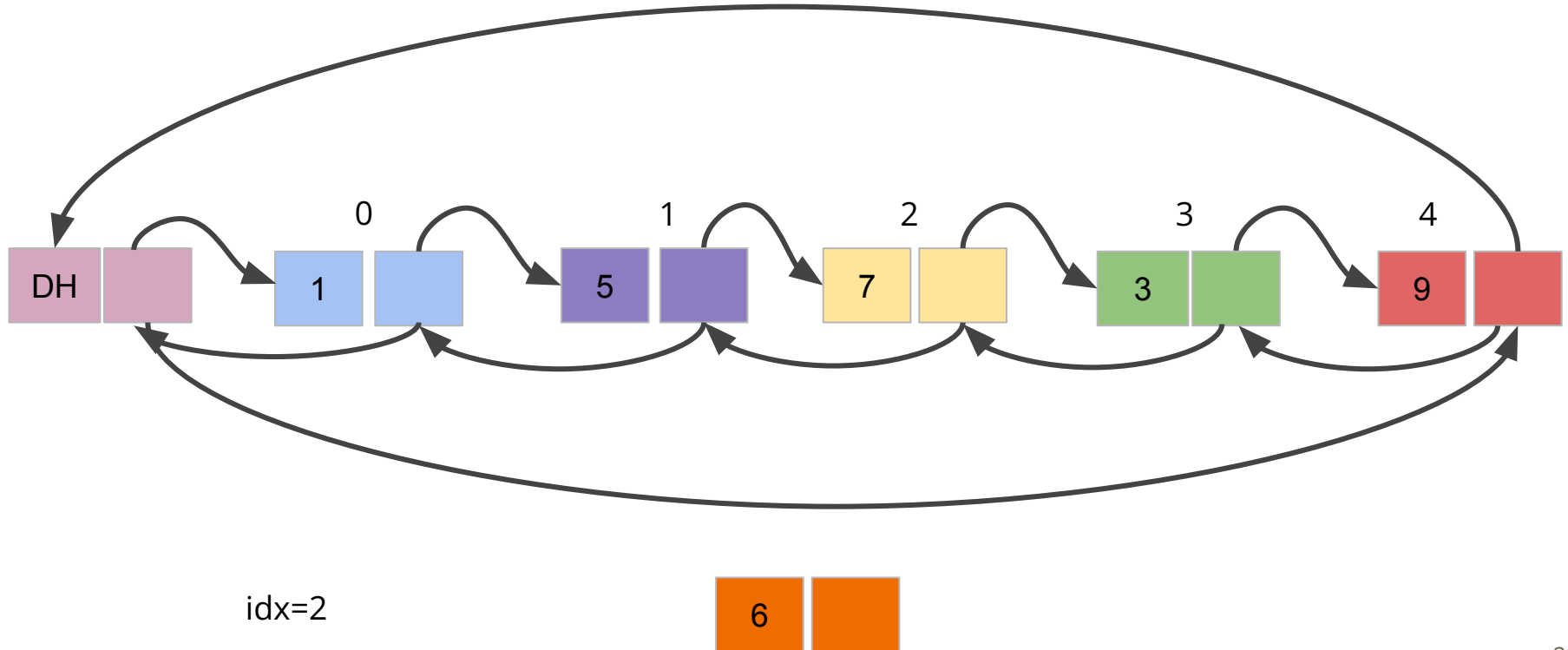
```
1 def nodeAt(dh, idx):
2     temp = dh.next
3     c = 0
4     while temp != dh:
5         if c == idx:
6             return temp
7         c += 1
8         temp = temp.next
9     return None # Invalid Index
```

Note that, if you want to count the number of total nodes, you should ignore the dummy head too.

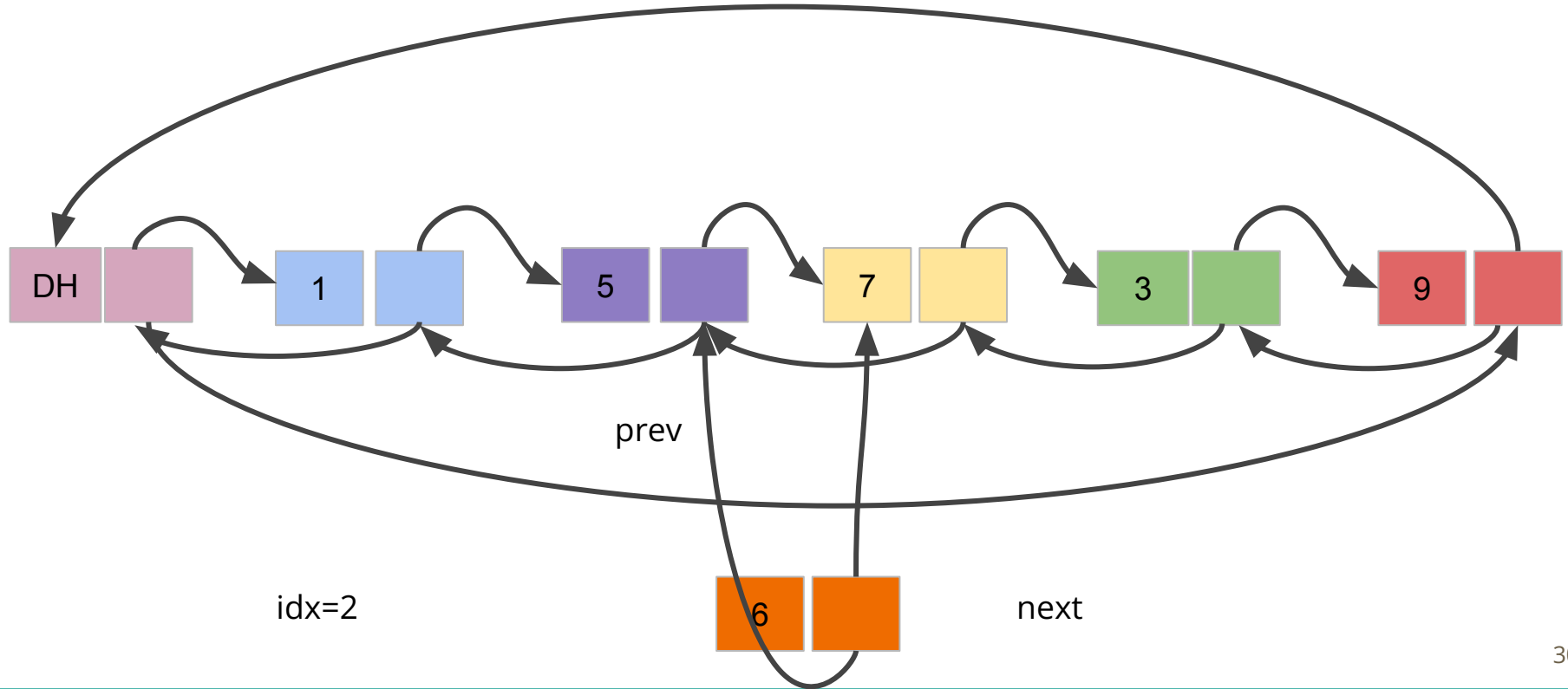
Insertion

To insert a new node in the list, you need the reference to the predecessor to link in the new node. Unlike for a singly-linked linear list, there is no "special" case here, since there is always a valid predecessor node available, thanks to the dummy head.

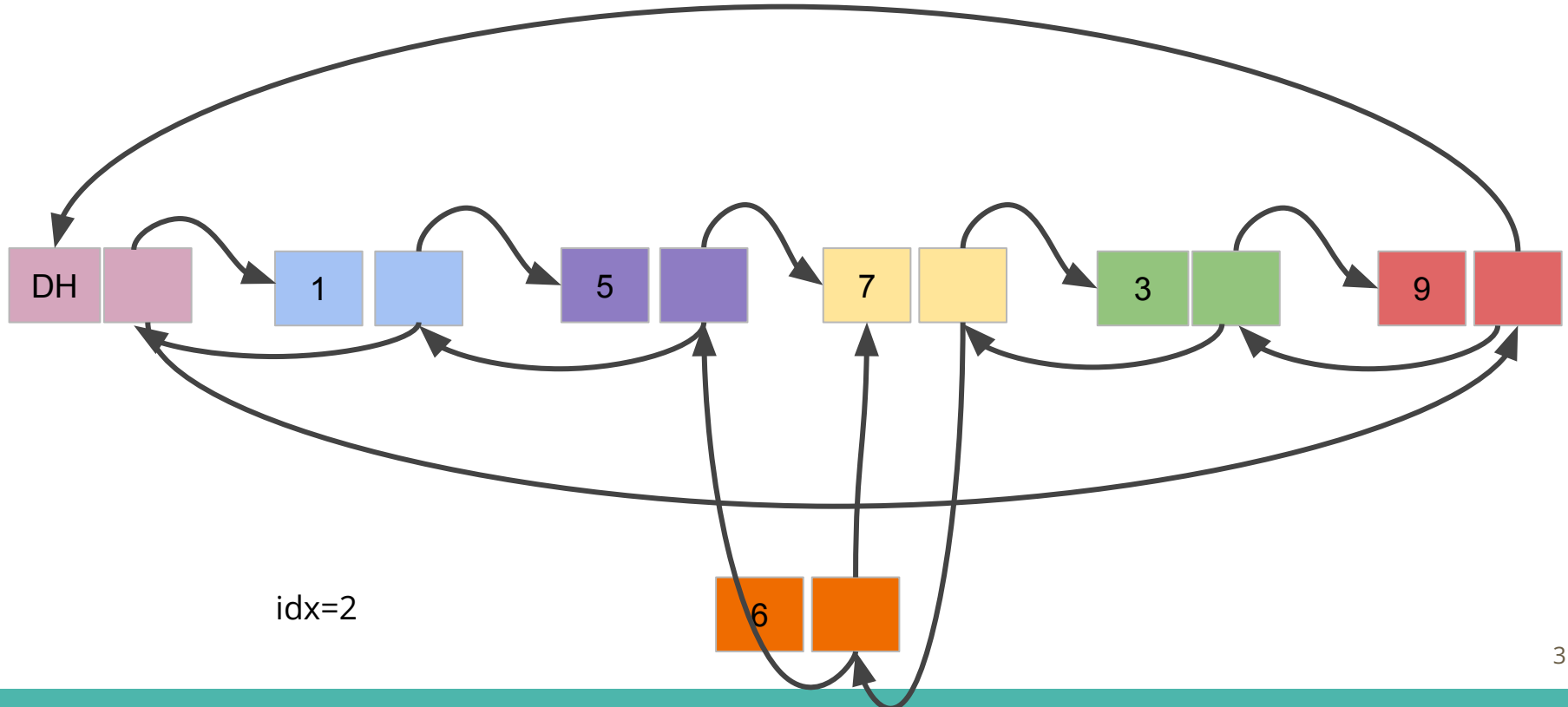
Insertion



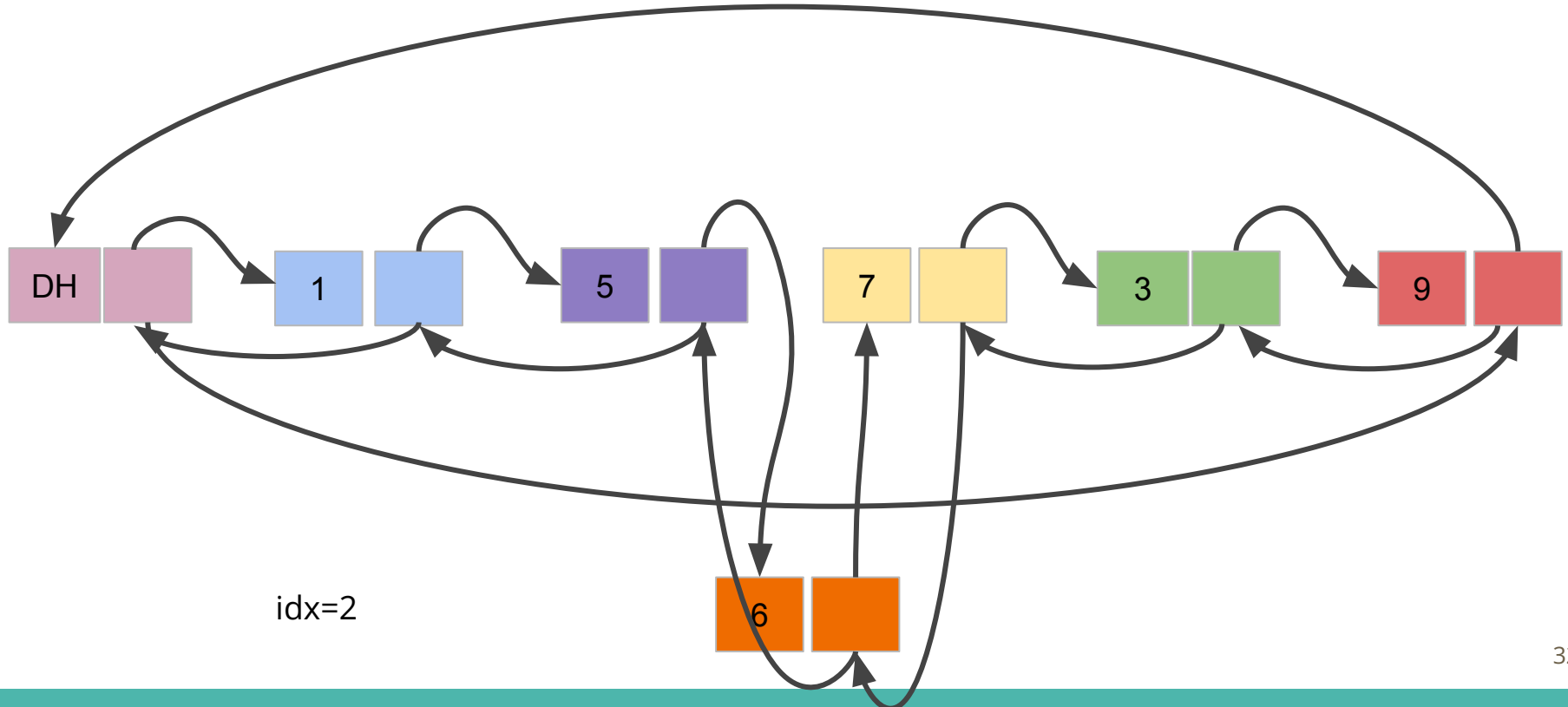
Insertion



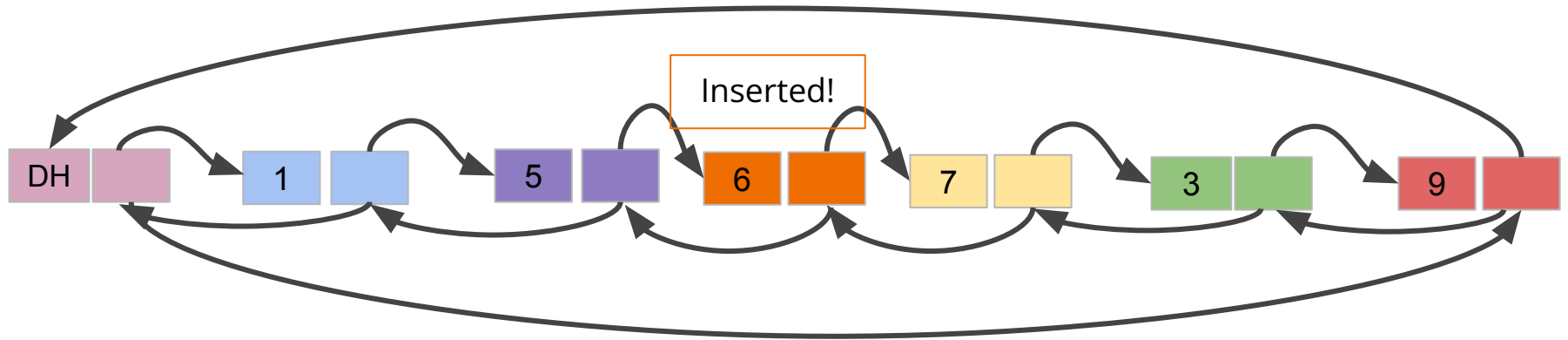
Insertion



Insertion



Insertion



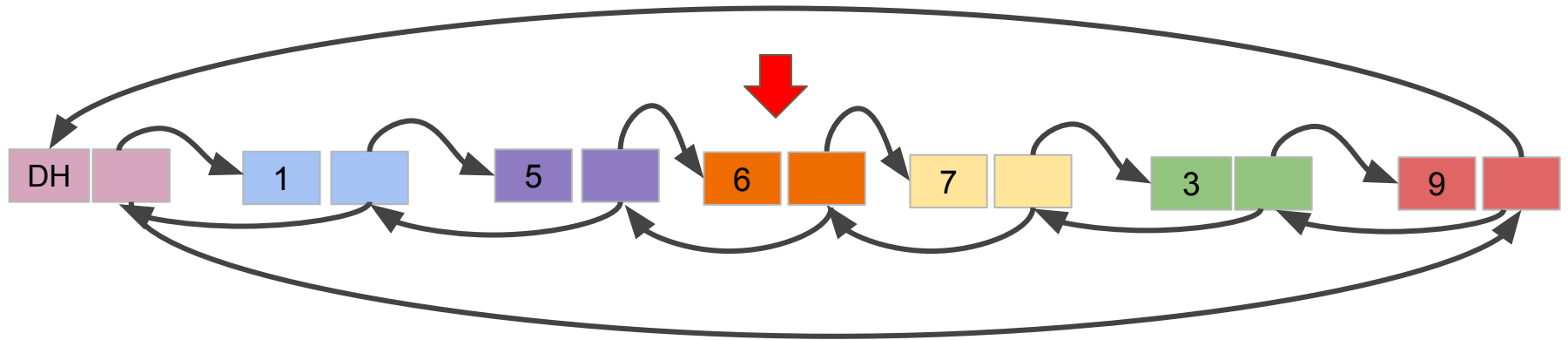
Insertion

```
1 def insertion(dh, elem, idx):
2     # Assuming the idx is valid
3     node_to_insert = DoublyNode(elem, None, None)
4     indexed_node = nodeAt(dh, idx) # Retriving the node at that index
5     prev_node = indexed_node.prev # There will always be a previous node
6     # Change the connection
7     # Observe that no special case is needed
8     node_to_insert.next = indexed_node
9     node_to_insert.prev = prev_node
10    prev_node.next = node_to_insert
11    indexed_node.prev = node_to_insert
```

Removal

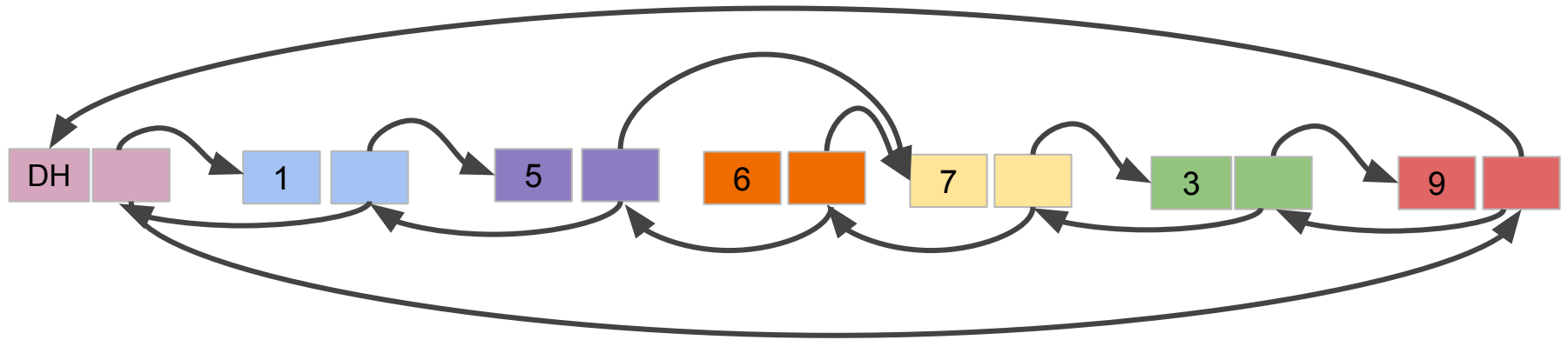
Removing an element from the list is done by removing the node that contains the element. Just like inserting a new node in a list, removing requires that you have the reference to the predecessor node. Since we're using a doubly-linked list, finding a predecessor of a node is trivial — it's `n.prev`. And thanks to the dummy head, there is no "special" case here as well.

Removal



idx=2

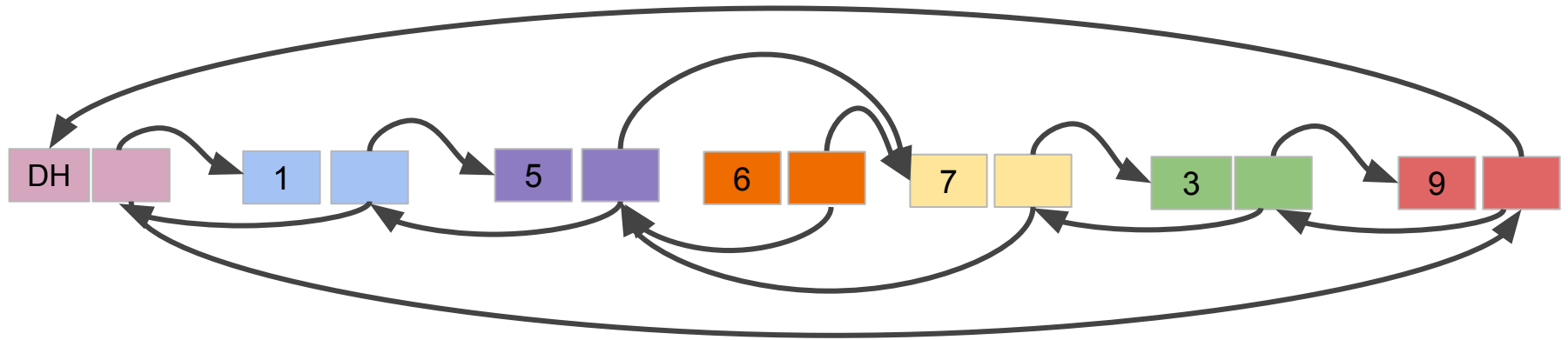
Removal



idx=2

We bypass the node to be removed

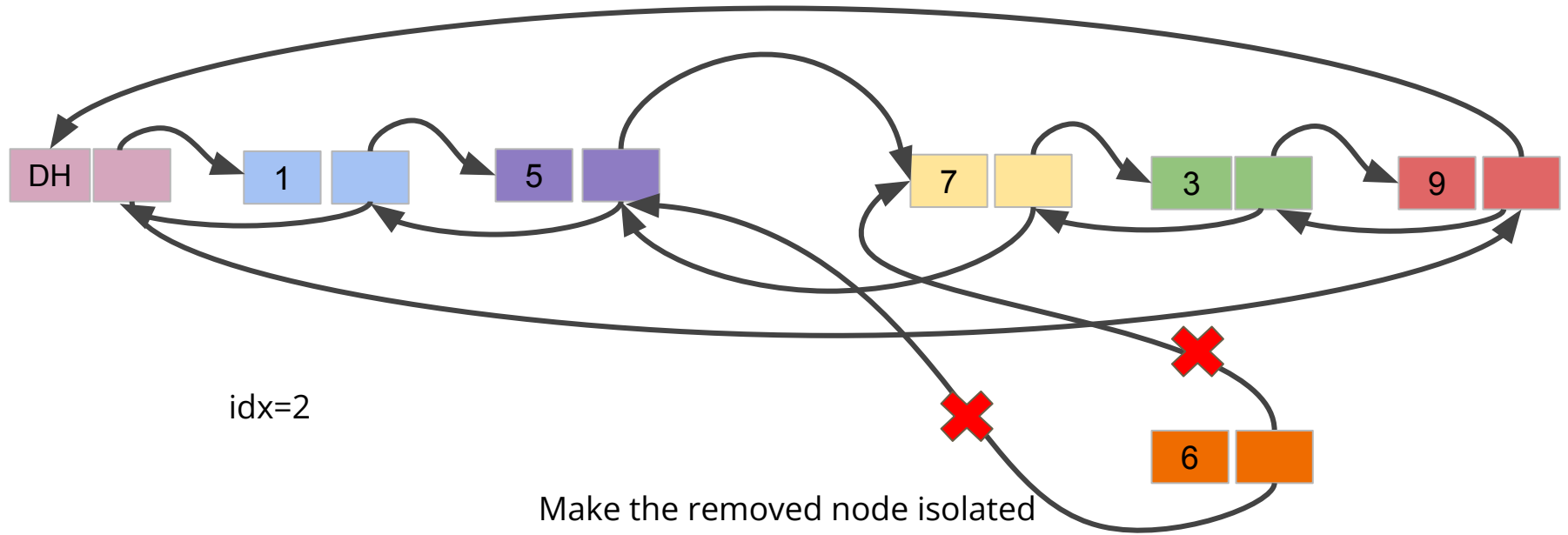
Removal



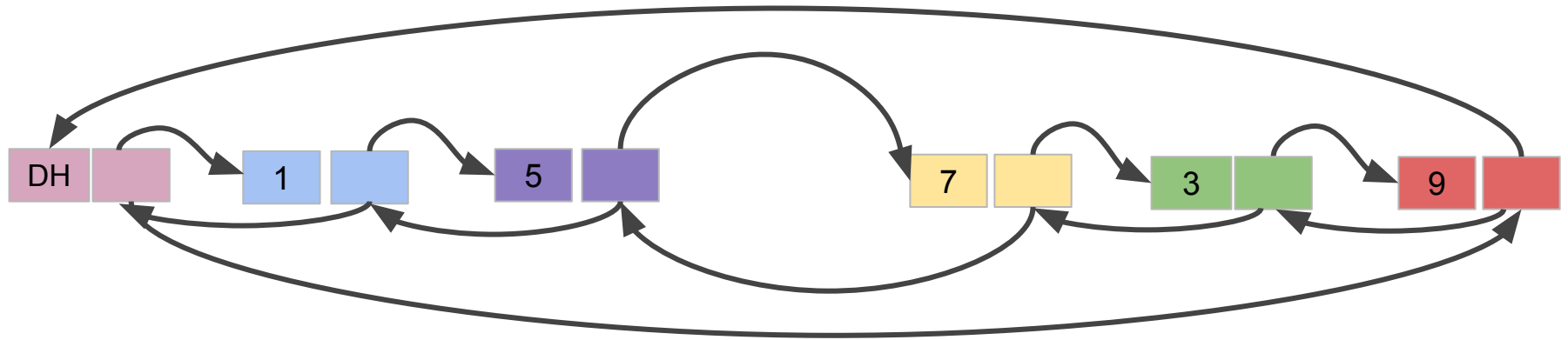
idx=2

We bypass the node to be removed

Removal



Removal



`idx=2`

Make the removed node isolated

6

Removed!

Removal

```
1 def removal(dh, idx):
2     # Assuming the idx is valid
3     node_to_remove = nodeAt(dh, idx)
4     prev_node = node_to_remove.prev
5     next_node = node_to_remove.next
6     # Change the connection
7     # No special case is needed
8     prev_node.next = next_node
9     next_node.prev = prev_node
10    node_to_remove.next = None
11    node_to_remove.prev = None
12    return node_to_remove.elem # Returning the removed element
```



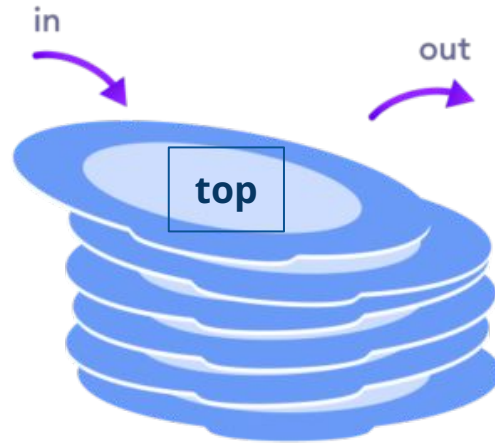
Stack



Stack Data Structure

- Store information, and access the more recently stored information first
- Maintains Last In First Out (LIFO) principle
- Example: Stack of plates, deck of cards, piles of books, piles of money etc

Stack Data Structure



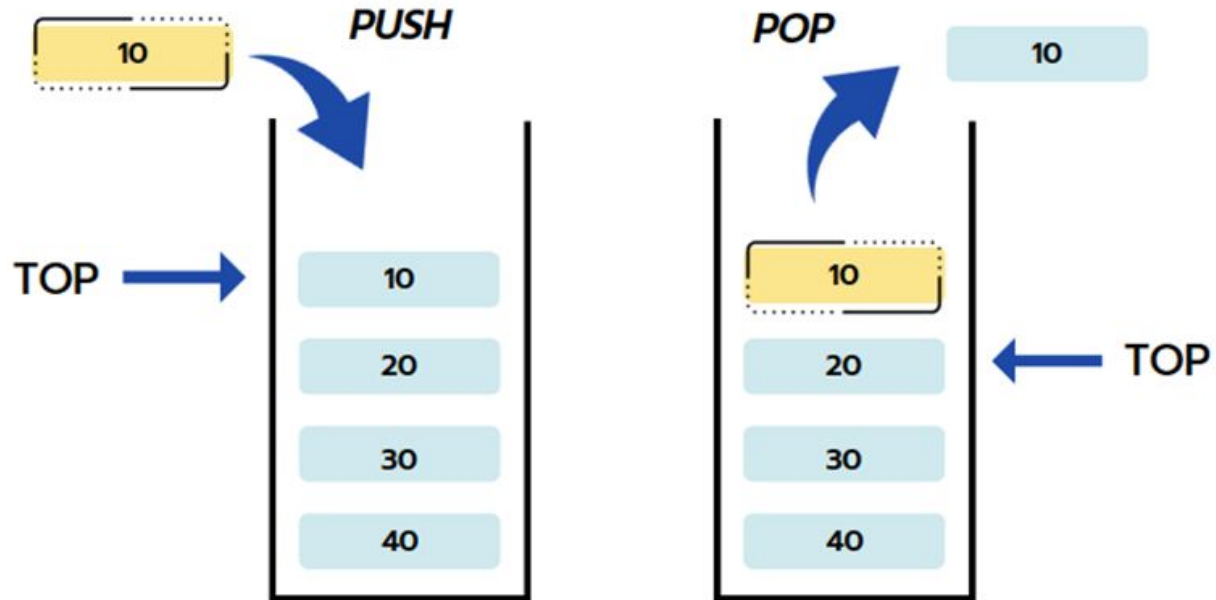
Stack is a restricted ordered sequence in which we can only add to and remove from one end — the top of the stack

Stack Data Structure

You are not, strictly speaking, allowed to

- Add to the middle of the stack
- Remove a plate from the middle of the stack
- The only plate that can be taken out of the stack is the most recently added one
- A stack is thus a "last in, first out" (LIFO) data structure.

Stack Data Structure



STACK DATA STRUCTURE

Stack Operations

Three basic stack operations are:

push(obj): adds obj to the top of the stack ("overflow" error if the stack has fixed capacity, and is full)

pop: removes and returns the item from the top of the stack ("underflow" error if the stack is empty)

peek: returns the item that is on the top of the stack, but does not remove it ("underflow" error if the stack is empty)

Stack Applications : Call Stacks

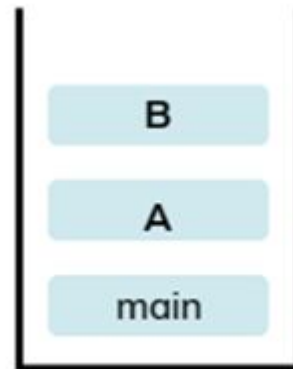
- Function calls are stored as stack records
- Whenever your C program calls a function, the C compiler creates a so-called stack record that contains important information about that function, such as storage for its parameters, its return value, and its local variables.
- The C compiler pushes this stack record onto something called a call stack.
- The call stack is organized so that the stack record of the most recently called function is on top of the stack, and the stack record of the least recently called function, which in the case of C is main, is at the bottom of the stack.

Stack Applications : Call Stacks

- For example, if the main calls A, which in turn calls B, then the call stack would be:

Corresponding code:

```
def B():  
    print("B")  
def A():  
    B()  
    print("A")  
def main():  
    A()  
    print("Main")  
if __name__ == "__main__":  
    main()
```

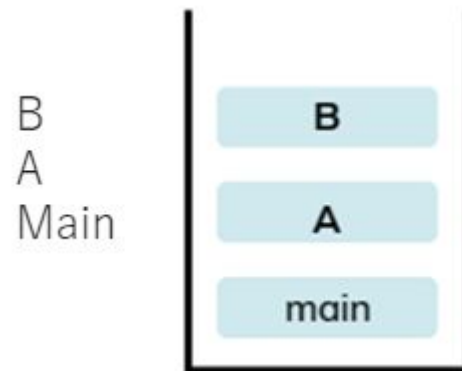


Stack Applications : Call Stacks

- For example, if the main calls A, which in turn calls B, then the call stack would be:

Corresponding code:

```
def B():  
    print("B")  
def A():  
    B()  
    print("A")  
def main():  
    A()  
    print("Main")  
if __name__ == "__main__":  
    main()
```



Stack Applications : Reverse

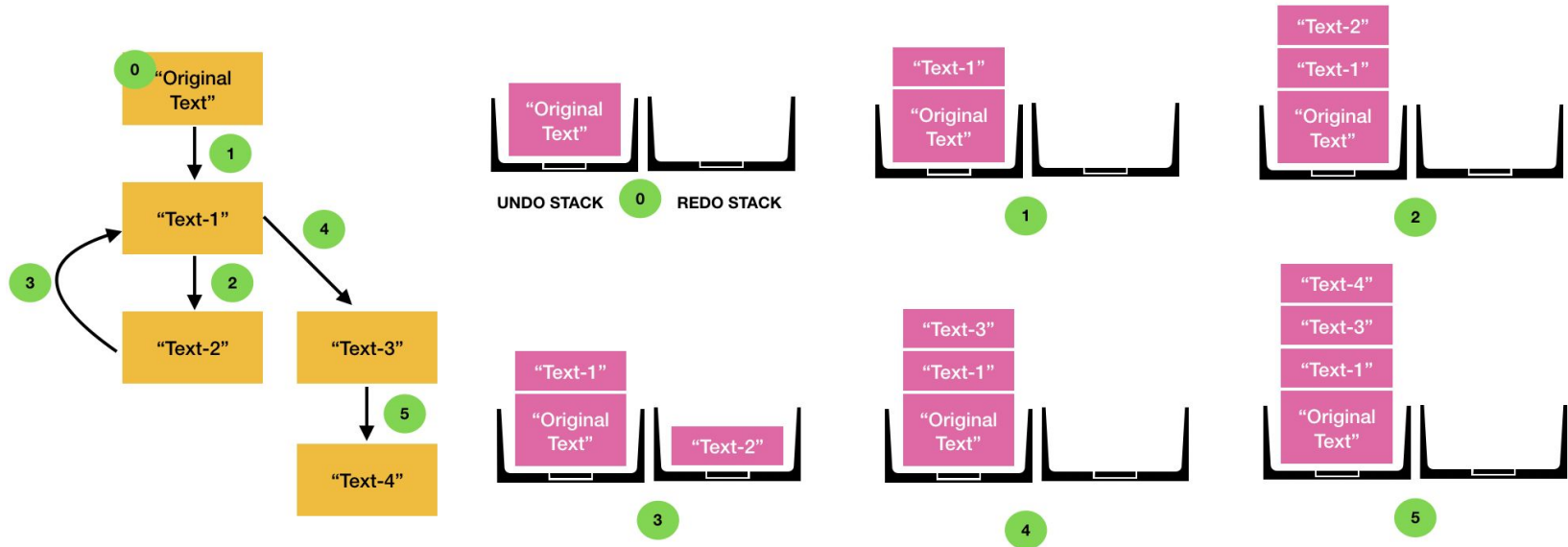
- The simplest application of a stack is to reverse a word. You push a given word to stack – letter by letter – and then pop letters from the stack.

```
begin with an empty stack and an input stream.  
while there is more characters to read, do:  
    read the next input character;  
    push it onto the stack;  
end while;  
while the stack is not empty, do:  
    c = pop the stack;  
    print c;  
end while;
```

Stack Applications : Undo

- An "undo" mechanism in text editors
- Accomplished by keeping all text changes in a stack
- Popping the stack is equivalent to "undoing" the last action
- Similar: going back pages in a browser using the back button

Stack Applications : Undo

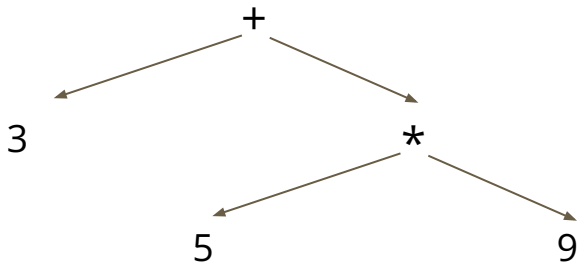


This is just for visualization purposes, no need to memorize

Stack Applications : Expression Evaluation

- When an arithmetic expression is presented in the postfix form, you can use a stack to evaluate it to get the final value.
- For example: the expression $3 + 5 * 9$ (which is in the usual infix form) can be written as $3\ 5\ 9\ *\ +$ in the postfix.

Postfix: left subtree -> right subtree -> root



Stack Applications : Expression Evaluation

- Postfix form removes all parentheses and thus all implicit precedence rules
- **Task:** write the postfix form corresponding to the following in-fix form:
 $((3 + 2) * 4) / (5 - 1)$

Stack Applications : Expression Evaluation

- Postfix form removes all parentheses and thus all implicit precedence rules
- **Task:** write the postfix form corresponding to the following in-fix form:

$((3 + 2) * 4) / (5 - 1)$

Answer: $3\ 2\ +\ 4\ *\ 5\ 1\ -\ /\$

Stack Applications : Expression Evaluation

You can now design a calculator for expressions in postfix form using a stack.

Stack Applications : Expression Evaluation

```
begin with an empty stack and an input stream (for the expression).  
while there is more input to read, do:  
    read the next input symbol;  
    if it's an operand,  
        then push it onto the stack;  
    if it's an operator  
        then pop two operands from the stack;  
        perform the operation on the operands;  
        push the result;  
end while;  
// the answer of the expression is waiting for you in the stack:  
pop the answer;
```

Stack Applications : Expression Evaluation

Let's apply this algorithm to to evaluate the postfix expression $3\ 2\ +\ 4\ *\ 5\ 1\ -\ /$ using a stack:

Stack Applications : Expression Evaluation

Stack	Expression
 --- (empty)	3 2 + 4 * 5 1 - /
3 ---	2 + 4 * 5 1 - /
2 3 ---	+ 4 * 5 1 - /
5 ---	4 * 5 1 - /
4 5 ---	* 5 1 - /
20 ---	5 1 - /
5 20 ---	1 - /

Stack Applications : Expression Evaluation

```
| 1 |  
| 5 |  
| 20 |  
---
```

- /

```
| 4 |  
| 20 |  
---
```

/

```
| 5 |  
---
```

(finished. the result is on top of the stack)

Stack Applications : Expression Evaluation

Convert $(1+2)*(4-3) + 1$ into the postfix format and evaluate the postfix expression using stack

Stack Applications : Parentheses Matching

- In many editors, it is common to want to know which left parenthesis (or brace or bracket, etc) will be matched when we type a right parenthesis
- Clearly we want the most recent left parenthesis.
- Therefore, the editor can keep track of parentheses by pushing them onto a stack, and then popping them off as each right parenthesis is encountered.
- We often have expressions involving "()[]{}" that require that the different types of parentheses are balanced.

Stack Applications : Parentheses Matching

The following are properly balanced:

- $(a(b + c) + d)$
- $[(a\ b)(c\ d)]$
- $([a\{x\ y\}b])$

The following are not properly balanced:

- $(a(b + c) + d)$
- $[(a\ b)(c\ d)]$
- $([a\{x\ y\}b])$

Stack Applications : Parentheses Matching

The Algorithm to follow

```
begin with an empty stack and an input stream (for the expression).
while there is more input to read, do:
    read the next input character;
    if it's an opening parenthesis/brace/bracket "(" or "{" or "["
        then push it onto the stack;
    if it's a closing parenthesis/brace/bracket ")" or "}" or "]"
        then pop the opening symbol from stack;
        compare the closing with opening symbol;
        if it matches
            then continue with next input character;
        if it does not match
            then return false;
end while;
// all matched, so return true
return true;
```

Stack Applications : Backtracking

This is a process when you need to access the most recent data element in a series of elements. Think of a labyrinth or maze - how do you find a way from an entrance to an exit?



Stack Applications : Backtracking

- Once you reach a dead end, you must backtrack. But backtrack to where? to the previous choice point.
- Therefore, at each choice point you store on a stack all possible choices.
- Then backtracking simply means popping a next choice from the stack.

Stack Applications : Language Processing

- Space for parameters and local variables is created internally using a stack (activation records).
- The compiler's syntax check for matching braces is implemented by using stack.
- Support for recursion